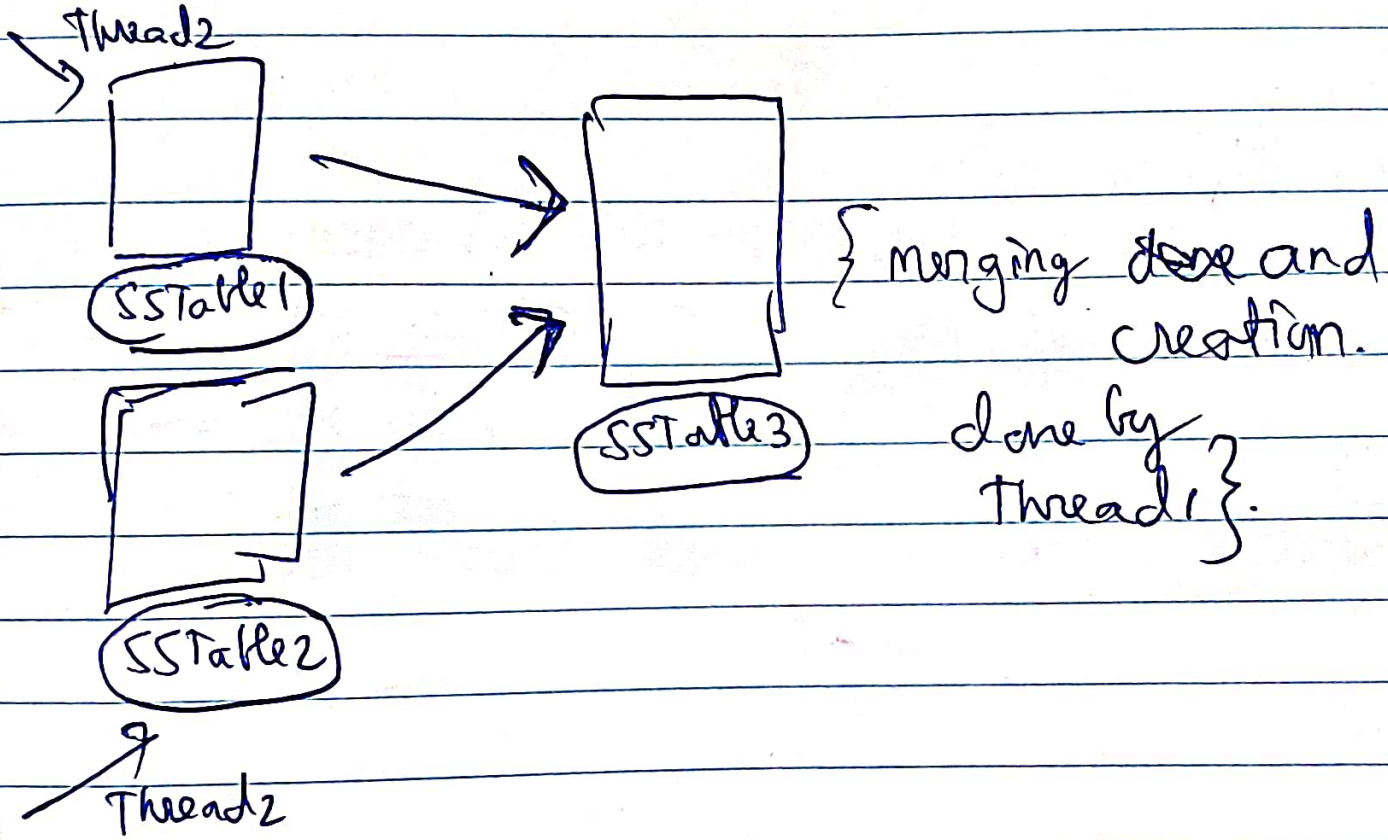
(Revision chapter 3)

How compaction works in Cassandra.

Thread1 → thread is responsible for making and merging and compaction.



Thread2 → responsible for routing read requests to appropriate SSTable.

Then when final compaction is completed, then traffic is routed to new SSTable

(For searching)

In memory hashmap is present for each Segment File on disk RAM

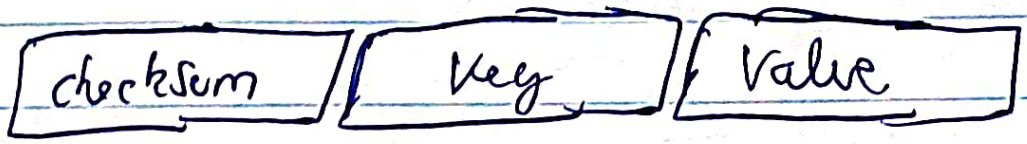
In case of server failure, this in-memory Hashmap will be gone from the server, and in that case it can be constructed again from the disk as well.

(Bitcask)

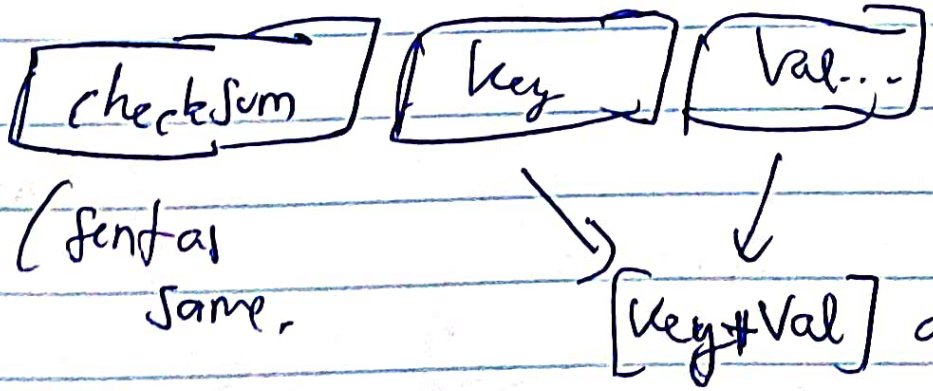
It does an optimisation by storing this Hashmap itself in the disk in Bitcask

[How to avoid partial writting & corrupt data writting.]

When (key, value) pair is getting written,



A new checksum is sent



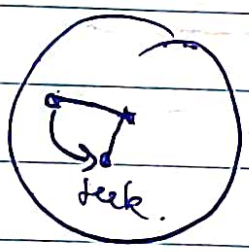
another checksum is generated to check if its the same or not

(Concurrency Control) → if multiple writer threads are there, so who appends at the end, hence we only given 1 writer thread & introduce locks.

Multiple Reader threads.

Q) Why append only is faster than Random Access?

Writing to disk just at the end is much faster than ~~appending~~



Then first go and do a seek and then find the location and then overwrite

Also in overwriting in a disk it can overwrite a record below it, if the new value is large

(key1: "xyzABC"
key2: "-----...") → If we try to overwrite over here for key1 with some very big value.

key1 = xyz.....123

↓

(key1: "xyzABC-----
123y: "-----") → this will overwrite the record below.

In file we don't shift the characters.

(Limitations)

HashTables should be small enough to fit in-memory

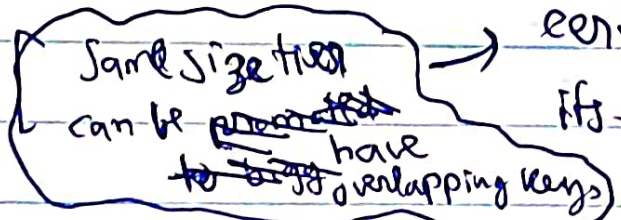
Range queries will require linear time.

Reads are slow because we are searching for the key in memtable, then in most recent SSTable,

Hence we keep Bloom filters to decide ~~whether~~ where the key might be present

Different types of compaction strategy we use.

→ Sizetiered → if after compaction size of (Cassandra). SSTable grows beyond a certain size then promote its level



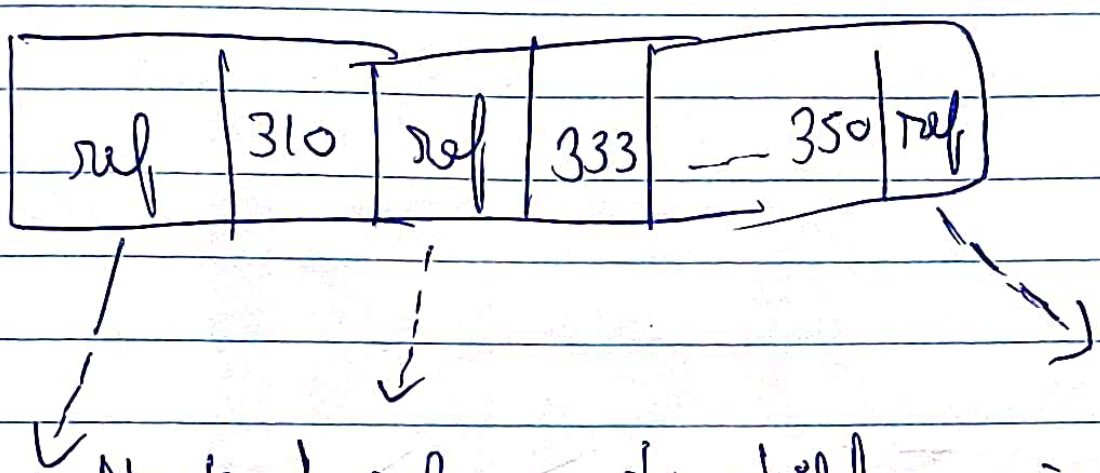
→ Leveled Compaction → SSTables in the same (RocksDB). level are compacted when growing high in number

In one level there will be no overlapping keys.

(B trees)

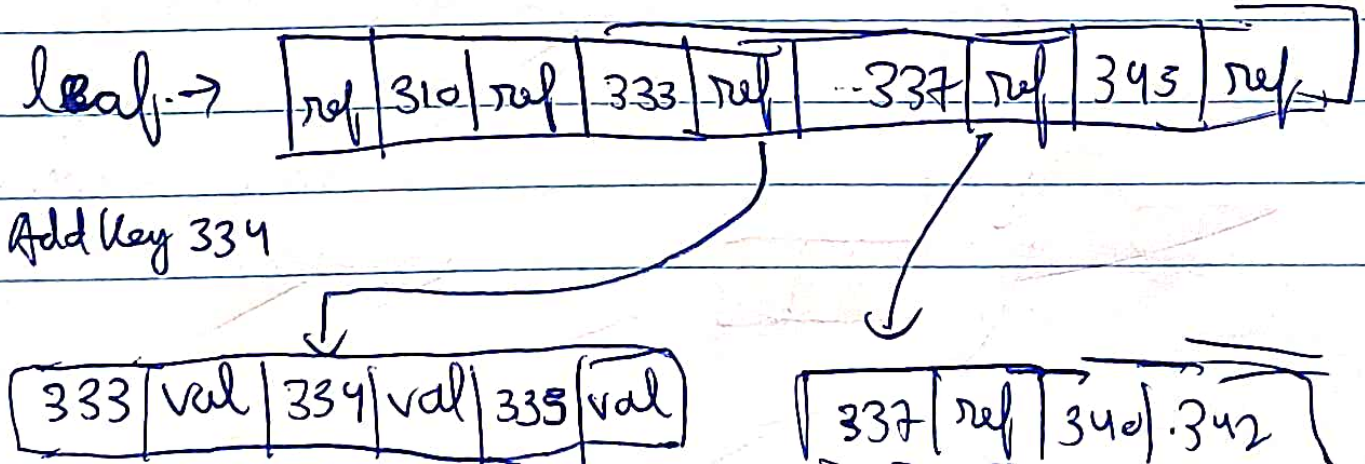
LSM trees the SSTables or the segment files can be of variable sizes. Initially of fixed size but upon merging different sizes.

(B-tree) fixed size 4KB pages.



Number of references to child page in one page of B-tree is called branching factor.

In case of B-trees what can happen is that you can have a leaf page split into two page



Problem in B-trees.

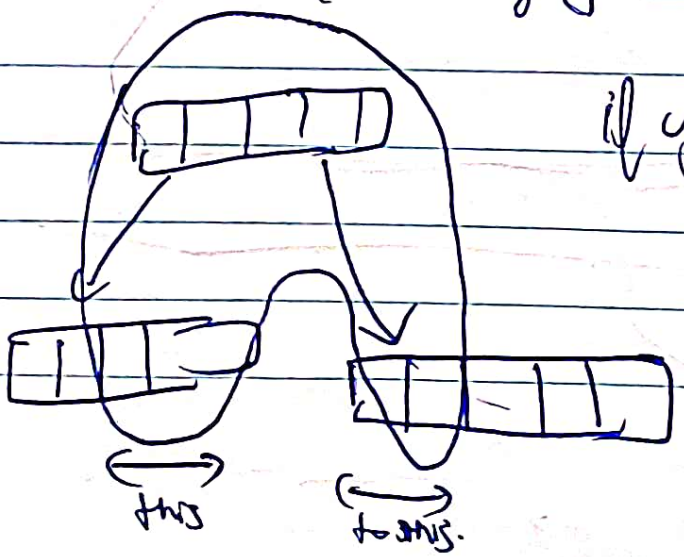
Like as in the previous page we saw how spliting of one page into two

During that if the system crashes, then RAM index is lost.

What we might have to do is Keep a WAL to recovery with DBs do.

(Concurrency)

Since in B-trees the actual address/ location can be modified so we need to apply locks (latches) properly accordingly



if you want to do range queries in B-tree, both up and down traversal has to be done

This is where B+trees comes into place

(Why Btrees have high write amplification than LSM trees?

LSM tree → Merges SSTables into single one, although involves frequent rewrite. but efficient due to sequential nature of writes

Btrees → node splits can lead to propagating writes to lower parts of the tree.

(Rebalancing)

[No Rebalancing in LSM trees,]
[B-trees have rebalancing.]

(***) (Fragmentation)

(LSM trees)

Since writes are happening in a sequential fashion hence minimum fragmentation.

(B-trees) → Random writes ~~[illegible]~~ higher fragmentation.

(***) In today's date many SSDs are internally using log structured append only instead of random writes to support faster (SSDs).

LSM Trees	B trees.
→ low write amplification, lesser compaction	→ higher write amplification, Random access, Space unused, causing fragmentation.
→ Compaction thread, Memtable flushing thread, User access thread. (Complex System)	→ Each key present only in 1 place, good for Transactional System

One strategy, is to store the whole row in the index, called clustered index.

But that would lead to great memory and write,
Better to use middle somewhere called covering index.